Estructuras de Datos Dinámicas

Estructuras de datos que crecen y se achican durante la ejecución de un programa.

Algunos ejemplos de estructuras dinámicas:

- ***Arrays Dinámicos***
  - Se les asigna memoria recién al ejecutar su declaración.
- ***Listas enlazadas***
  - Colección de elementos dispuestos uno a continuación del otro.
  - Permiten inserciones y borrados en cualquier parte de la lista.
- ***Pilas***
- ***Colas***
- ***Matrices Dinámicas***
  - Vector dinámico de vectores dinámicos

Asignación Dinámica de la Memoria

Permite hacer un *uso más eficiente de la memoria*, evitando que las variables y estructuras ocupen espacio cuando no se usan.

El espacio de las variables asignadas dinámicamente se crea *durante la ejecución del programa*, y no como en el caso de las variables estáticas, cuyo espacio se asigna en *tiempo de compilación*.

La asignación se realiza con la instrucción *new* y el *tipo de dato* para el que se quiere reservar memoria.

$$Ptr = new\ int;$$

La *memoria dinámica* se obtiene y libera *durante la ejecución del programa*.

Cuando se necesita crear una variable dinámica se deberá solicitar memoria dinámica con el operador:

- ***new***
  - Solicita reserva de memoria de un número de bytes.
  - Retorna un puntero de tipo * $void$
    - Es un puntero a la dirección del bloque asignado de memoria.
  - Si no hay memoria disponible retorna ***NULL***

La asignación de memoria con el operador *new* se simplifica mucho ya que con una única expresión ya se *calcula el tamaño*, se *realiza la asignación* y se hace la *conversión de tipo*.

Las *variables dinámicas* se acceden exclusivamente por *punteros*. No tienen un identificador asociado.

Cuando se necesita liberar la variable dinámica, se deberá usar el operador:

- ***delete***
  - Realiza dos acciones
    - 1. Destruye la variable dinámica
    - 2. Libera la memoria dinámica reservada para dicha variable
    - Indirectamente al destruir la variable dinámica, el puntero que apuntaba a esta queda con un valor no especificado
  - El argumento para delete es un puntero a la dirección de memoria que se quiere liberar.

delete : libera un único objeto

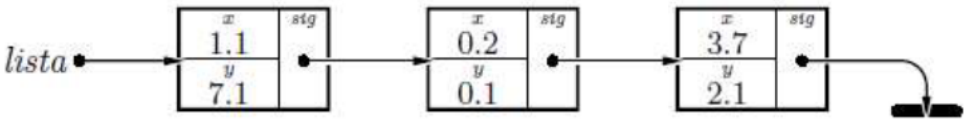
delete [ ] : libera un vector dinámico

Si se le pasa un puntero nulo a delete no ocurre nada, por esta razón se recomienda asignar ***NULL*** al puntero inmediatamente después de usar delete, para *evitar que el puntero pueda ser usado como argumento otra vez* para delete.

Estructuras Auto-Referenciadas

- Estructuras que tienen un puntero a una estructura de un mismo tipo.
- Pueden ser *enlazadas* para formar estructuras de datos como *listas*, *colas*, *pilas* y *árboles*.
- Terminan con un puntero ***NULL***
- *proximoPtr* se denomina *enlace*.

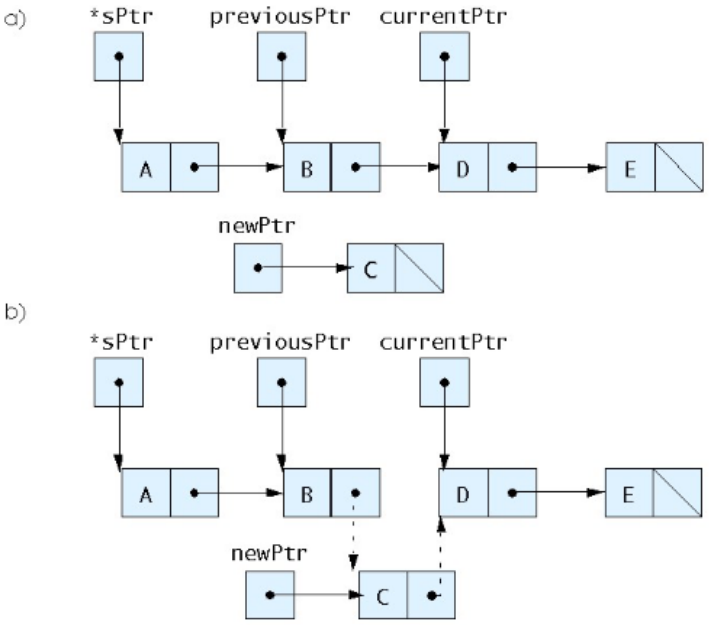
```
struct Nodo {
int dato;
Nodo  * proximoPtr;
};
```



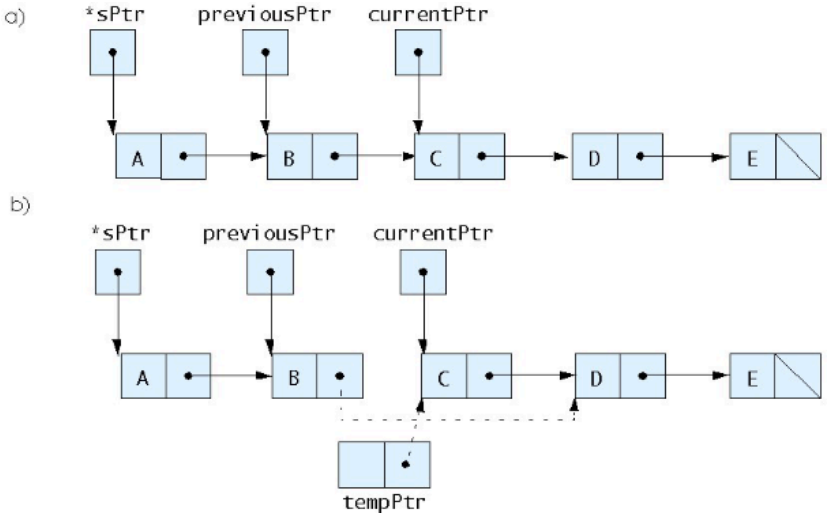
Listas Enlazadas

- Colección lineal de objetos autoreferenciados, llamados *nodos*.
- Conectados mediante punteos.
- Acceso mediante un *puntero maestro* al primer nodo de la lista.
- El puntero enlace del último nodo se pone a ***NULL*** para marcar el *final de la lista*.
- Los datos se almacenan *dinámicamente*.
- Los nodos se van creando a medida que se los necesita.
- Cuando usar listas enlazadas:
  - Cuando *no se conozca* por anticipado la cantidad de elementos de datos a representarse.
  - Cuando se necesiten inserciones y eliminaciones *rápidamente*.
    - La inserción y eliminación en arreglos ordenados lleva mucho tiempo porque hay que desplazar todos los elementos. En listas solo conectamos los nodos que quedaron sueltos separando el elemento eliminado.

Inserción de un nuevo nodo

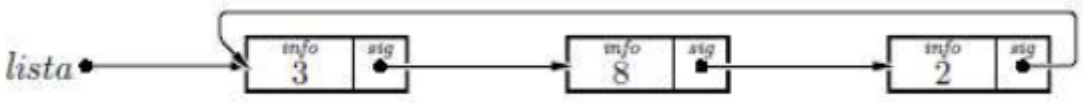


Eliminación de un nodo

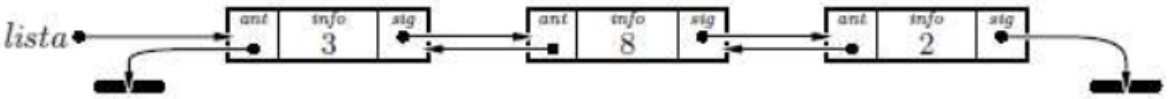


Otros tipos de Listas:

- ***Listas Circulares***



  - Lista enlazada simple en la que el último nodo referencia al primero
  - No hay un principio o fin definidos
  - Ningún nodo está enlazado a ***NULL***
  - El puntero maestro se usa para reconocer un ingreso a la lista
- ***Listas Doblemente Enlazadas***



  - Cada nodo tiene un puntero al siguiente y otro puntero al anterior nodo
  - Utiliza más memoria
  - Puede recorrerse en *ambas direcciones*
  - Las operaciones *insertar* y *eliminar* utilizan *menor cantidad de instrucciones*